

Acessar

arineto1.github.io/ifpe2026

Informática Teórica

Revisão - Aula 3

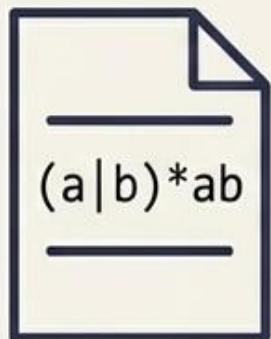
A Linha de Montagem da Lógica

Uma jornada estruturada: traduzimos a intenção humana num formato abstrato, refinamos a lógica para eliminar a ambiguidade e, finalmente, geramos código de alta performance.

01

O Contrato

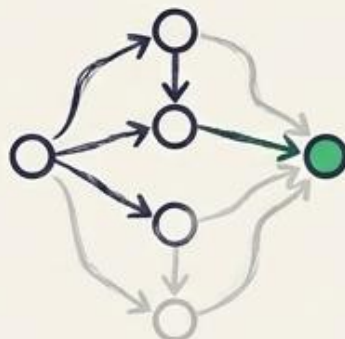
Expressão Regular
(ER)



02

O Esboço

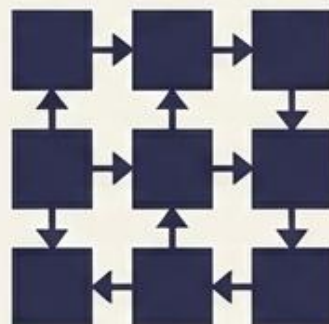
Autômato Não-Determinístico (AFN)



03

A Precisão

Autômato Determinístico (AFD)



04

A Máquina

Código de Execução

```
bool matches(string s) {  
    state = q0;  
    return true;  
    ...  
    return false;  
}
```

Passo 1: A Expressão Regular (O Contrato)

O que vem antes?

Qualquer combinação do nosso alfabeto {a, b}. O asterisco (*) indica repetição de zero ou infinitas vezes.

(a | b) * a b

90°

90°

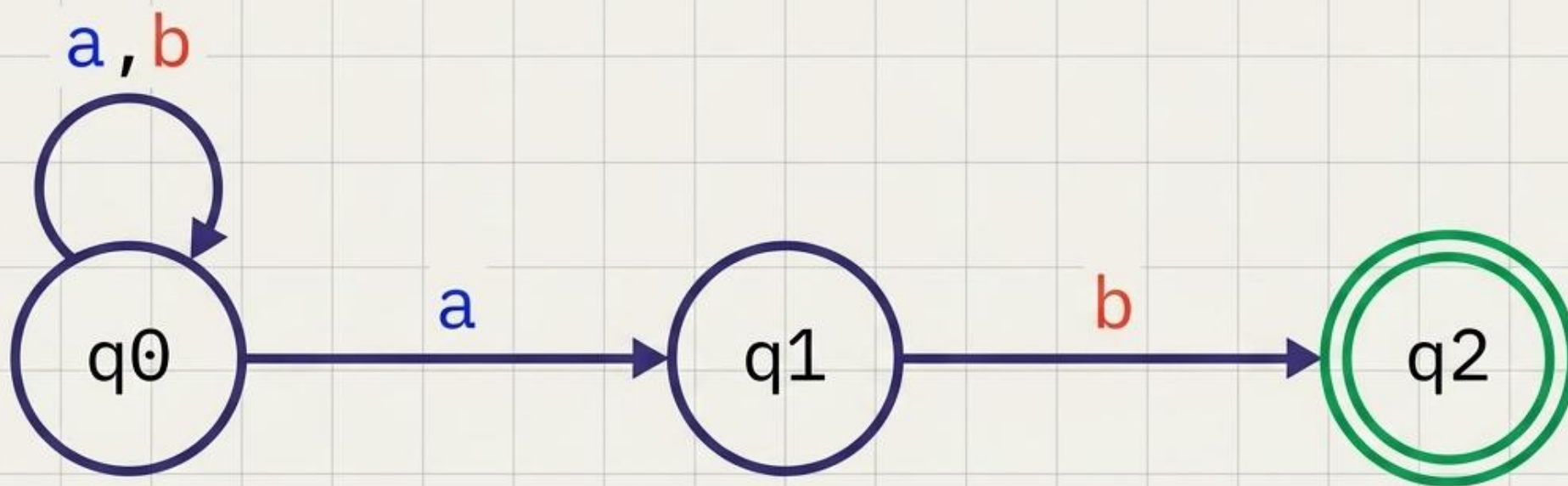
45°

O que vem no fim?

Obrigatoriamente a sequência exata de 'a' seguido de 'b'.

A Expressão Regular é a forma mais compacta de formalizar o nosso padrão.

Passo 2: O AFN (O Esboço Intuitivo)



A Intuição

O Autômato Não-Determinístico (AFN) pode tomar decisões simultâneas perante um único caráter.

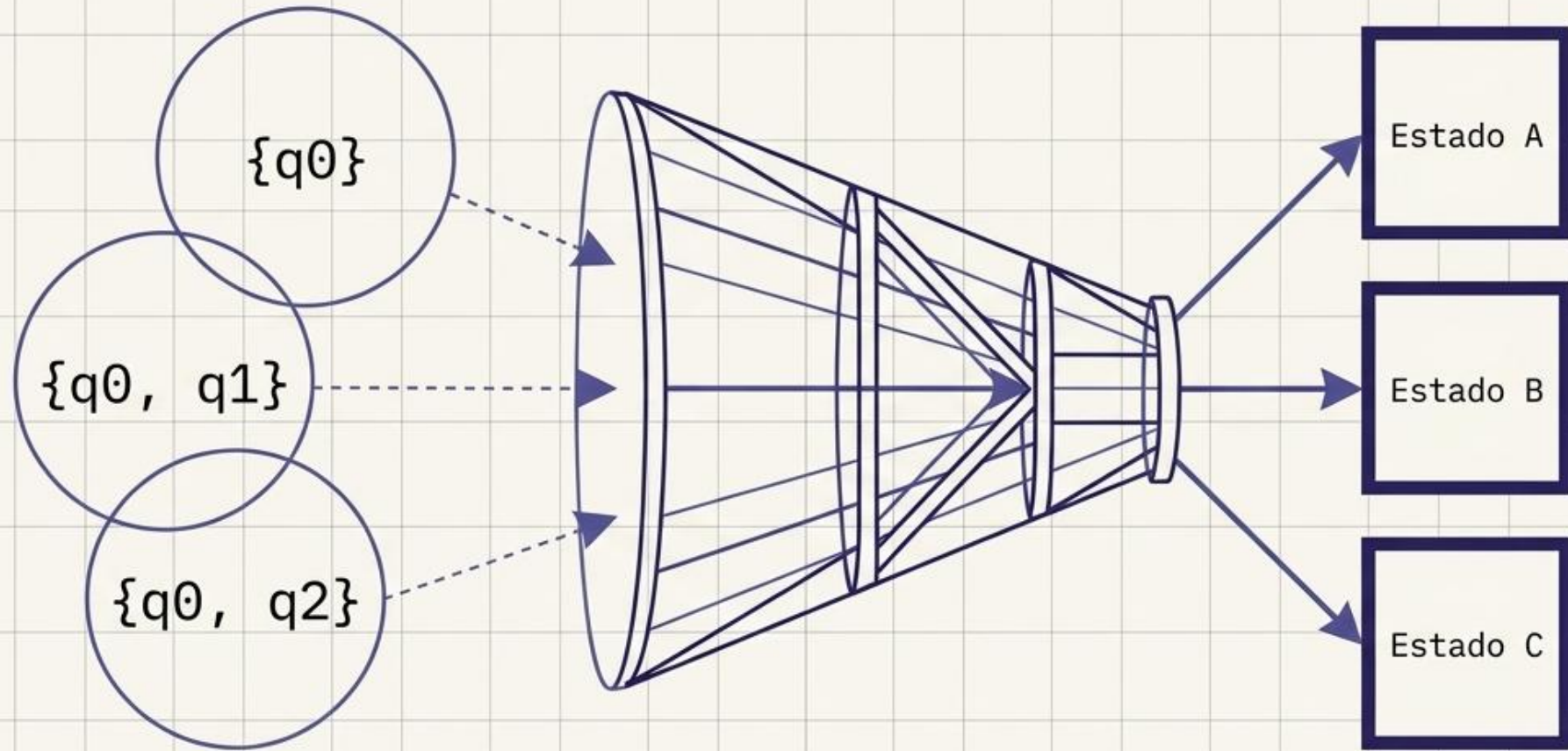
A Espera

O sistema fica preso no laço do estado q_0 a ler caracteres indefinidamente até encontrar um padrão útil.

A Aposta

Ao ler um 'a', o sistema pode continuar a esperar em q_0 OU apostar que é o início do sufixo e avançar para q_1 .

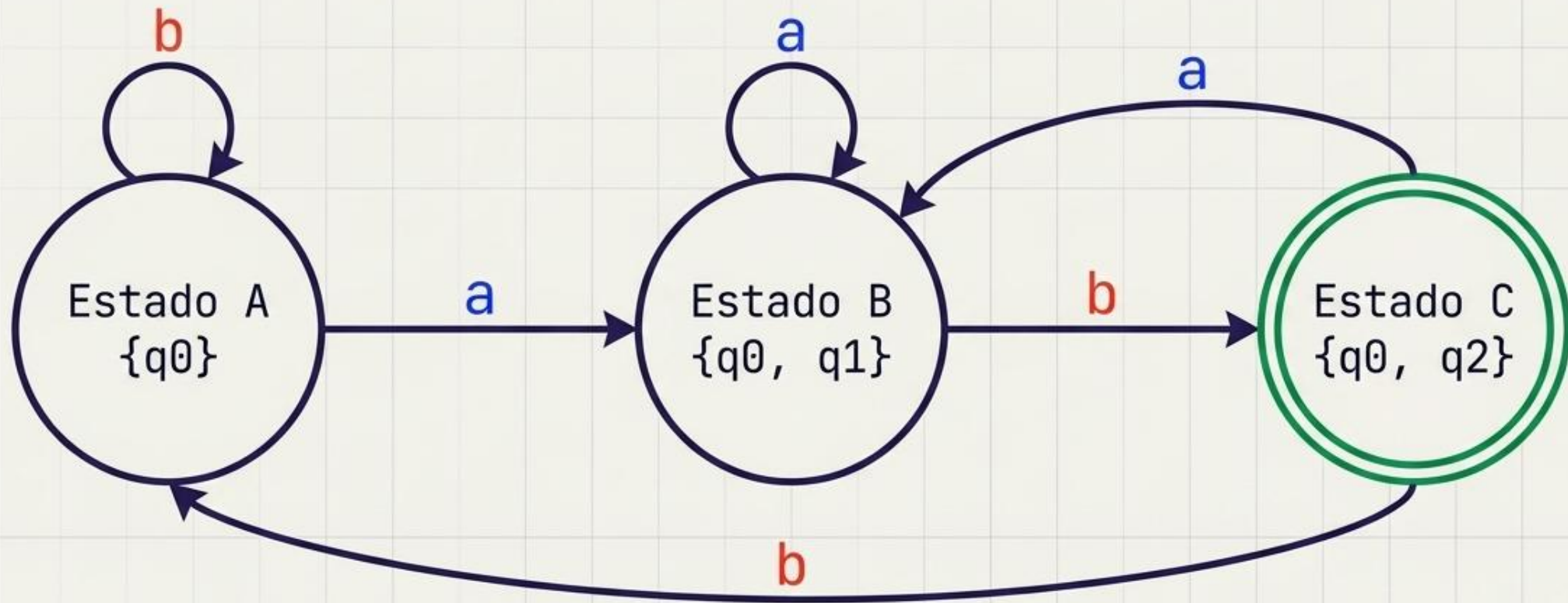
Passo 3: A Construção de Subconjuntos



A Engenharia de Precisão:

Para converter o AFN num Autómato Determinístico (AFD), agrupamos todas as posições onde a máquina poderia estar simultaneamente num único e impenetrável novo estado sólido. Agrupar incertezas cria certezas absolutas.

○ Grafo AFD (Engenharia de Precisão)



Sem Apostas:

Cada estado tem agora exatamente uma saída obrigatória para 'a' e uma saída obrigatória para 'b'. A ambiguidade foi matematicamente erradicada.

A Quebra de Padrão:

Se a máquina ler 'b' no Estado C, percebe que a sequência falhou e regressa instantaneamente à estaca zero (Estado A) sem hesitar.

Passo 4: O Código (A Máquina em Ação)

O AFD é puramente mecânico. Traduzimos o grafo desenhado numa simples matriz de regras, onde o estado atual e o carácter lido ditam o salto no código.

State Transition Matrix		
	Entrada 'a'	Entrada 'b'
Estado A	Estado B	Estado A
Estado B	Estado B	Estado C
Estado C	Estado B	Estado A

```
switch (estado_atual) {  
  case A:  
    if (c == 'a') estado_atual = B;  
    else estado_atual = A;  
    break;  
  case B:  
    if (c == 'a') estado_atual = B;  
    else estado_atual = C;  
    break;  
  case C:  
    if (c == 'a') estado_atual = B;  
    else estado_atual = A;  
    break;  
}
```


Informática Teórica

Aula 4

Expressões Regulares (ER)

Definição Técnica (O "Kit de Construção")

Formalmente, uma Expressão Regular é uma linguagem de descrição baseada em três elementos atômicos (as peças) e três operações (a cola):

As Peças (Base):

1. **Símbolos:** Qualquer caractere do alfabeto (ex: a , b , 0 , 1).
2. **Vazio (ϵ):** Representa uma "casa vazia" ou ausência de caractere.
3. **Conjunto Vazio ($\emptyset$):** Representa uma linguagem que não aceita nada.

As Operações de Base (União e Concatenação)

Definição Técnica (Escolha e Sequência)

As Expressões Regulares usam **duas regras fundamentais** para combinar símbolos:

- **União ($R_1 \cup R_2$ ou $R_1 | R_2$):** É a operação de **soma de conjuntos**. Se você tem a regra "A" e a regra "B", a união aceita qualquer palavra que venha de uma delas.
- **Concatenação ($R_1 \cdot R_2$):** É a operação de **junção**. Ela exige que uma palavra da primeira regra seja seguida imediatamente por uma palavra da segunda.

Exemplo Prático: O Menu da Lanchonete

Vamos criar uma **Expressão Regular** para um pedido simples:

- **As opções de bebida (União):** Café | Suco
- **O acompanhamento obrigatório (Concatenação):** Pão

A Expressão: (Café | Suco) Pão

A Expressão: (Café | Suco) Pão

O que o sistema aceita?

- Café Pão (**Aceito:** Escolheu café e seguiu a ordem).
- Suco Pão (**Aceito:** Escolheu suco e seguiu a ordem).
- Água Pão (**Rejeitado:** Água não estava no "OU").
- Pão Café (**Rejeitado:** A ordem da concatenação foi desrespeitada).

Exemplo Prático: O Menu da Lanchonete

Vamos criar uma **Expressão Regular** para um pedido simples:

- **As opções de bebida (União):** Café | Suco
- **O acompanhamento obrigatório (Concatenação):** Pão

A Expressão: (Café | Suco) Pão

regex101.com

regex101.com

^ - início da String

\$ - fim da String

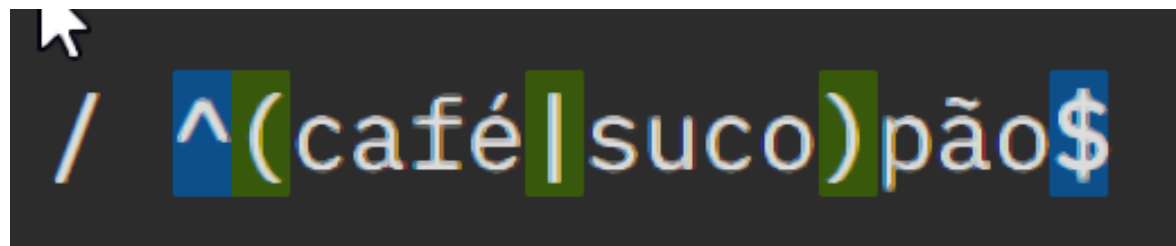
Exemplo Prático: O Menu da Lanchonete

Vamos criar uma **Expressão Regular** para um pedido simples:

- As opções de bebida (União): Café | Suco
- O acompanhamento obrigatório (Concatenação): Pão

A Expressão: (Café | Suco) Pão

regex101.com



```
/^(café|suco)pão$
```

O Fecho de Kleene (O Poder da Repetição)

Definição Técnica (O Acúmulo de Potências)

O operador **estrela** (*), ou Fecho de Kleene, representa a repetição de uma expressão regular zero ou mais vezes.

- Isso significa que R^* aceita a união de:
 - R^0 : A cadeia vazia (ϵ).
 - R^1 : A própria expressão R .
 - R^2 : A sequência RR .
 - ... e assim por diante, até o infinito.
- **Ponto Chave:** O fecho de Kleene **sempre** inclui a cadeia vazia, mesmo que o alfabeto original não a tenha.

Definição Simples e Contextualizada

Imagine o Fecho de Kleene como um "Botão de Repetir" que ficou travado.

Ele diz ao computador: o que estiver logo antes deste asterisco pode aparecer zero vezes (ou seja, sumir), uma vez ou milhões de vezes. Ele é o segredo para lidarmos com textos de tamanhos que não conhecemos de antemão.

Exemplo Prático: O "Grito" na Internet

Imagine que você quer criar um filtro para identificar quando alguém está "gritando" um "Ah!" com vários 'as' em um comentário.

- **O Problema:** O usuário pode digitar "Ah!", "Aah!", "Aaaaaah!", ou até apenas "h!" (se ele não digitar nenhum 'a').
- **A Expressão Regular:** `a* h !`

- A Expressão Regular: $a^* h !$

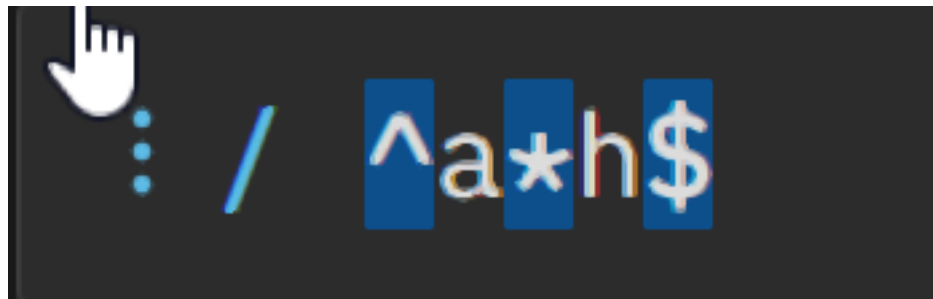
O que o sistema aceita?

- $h!$ (**Aceito**: O 'a' apareceu **zero** vezes).
- $ah!$ (**Aceito**: O 'a' apareceu **uma** vez).
- $aaaah!$ (**Aceito**: O 'a' apareceu **quatro** vezes).
- $oh!$ (**Rejeitado**: O símbolo 'o' não faz parte da regra).

- A Expressão Regular: `a* h !`

O que o sistema aceita?

- `h!` (**Aceito**: O 'a' apareceu **zero** vezes).
- `ah!` (**Aceito**: O 'a' apareceu **uma** vez).
- `aaaah!` (**Aceito**: O 'a' apareceu **quatro** vezes).
- `oh!` (**Rejeitado**: O símbolo 'o' não faz parte da regra).



regex101.com

O Teorema de Kleene (A Grande Equivalência)

Definição Técnica (A Via de Mão Dupla)

O Teorema de Kleene afirma que a classe das linguagens descritas por Expressões Regulares é exatamente a mesma classe das linguagens reconhecidas por Autômatos Finitos.

- **Sentido 1:** Se você tem uma **ER**, existe um **AFN/AFD** que a reconhece.
- **Sentido 2:** Se você tem um **AFN/AFD**, existe uma **ER** que o descreve.
- **Conclusão:** Linguagens Regulares = Linguagens de ER = Linguagens de Autômatos.

Exemplo Prático: O Detector de Binários

Imagine que você quer aceitar qualquer sequência de 0s e 1s.

- **A Expressão Regular (Texto):** $(0|1)^*$
- **O Autômato (Desenho):** Um único estado q_0 (inicial e final) com uma seta que volta para ele mesmo (loop) contendo os símbolos 0, 1.

Por que isso é útil?

Se você é um programador e escreve $(0|1)^*$ no seu código, o compilador usa o Teorema de Kleene para transformar automaticamente esse texto no desenho do autômato, que é o que o processador realmente sabe executar com velocidade.

Aplicações no "Mundo Real" (Regex)

Regex é a abreviação de **Regular Expressions** (ou Expressões Regulares, as mesmas que acabamos de discutir na teoria).

Enquanto na teoria os **automatos** e as **ERs** servem para provar o **que uma linguagem pode ou não fazer**.

Na prática da programação, **Regex** é uma ferramenta de busca e **manipulação de texto extremamente poderosa**.

Onde as Regex são usadas?

Quase todos os editores de código (VS Code, Notepad++) e linguagens de programação (Python, JavaScript, Java, PHP) suportam **Regex**. Elas servem para:

- **Validação:** O texto digitado é um e-mail válido? Tem o formato de uma data?
- **Busca:** Encontrar todas as palavras que começam com "A" e terminam com "o".
- **Substituição:** Trocar todos os pontos por vírgulas, mas apenas se estiverem entre números.
- **Extração:** Pegar apenas o DDD de uma lista de 500 números de telefone.

Exemplo de Reforço: O "Cérebro" do Compilador

Se você escrever inteiro **x = 10**; em C++, o compilador usa a Regex para entender:

inteiro → Opa, isso bate com a minha Regex de **Tipo de Dado**.

x → Isso bate com a minha Regex de **Identificador**.

10 → Isso bate com a minha Regex de **Literal Numérico**.

Dada a Expressão Regular $R = (a|b)^*c$, analise o "molde" e determine se as seguintes cadeias são **Aceites** ou **Rejeitadas**:

- A) abc
- B) aaababbbc
- C) c
- D) ab

regex101.com

Um sistema de e-mail quer bloquear mensagens que contenham gritos ou risos excessivos.
Crie Expressões Regulares para identificar:

- **Padrão A:** Pelo menos um "k" seguido de vários "k"s (ex: `kkk`, `kkkkkk`).
- **Padrão B:** A palavra `URGENTE` ou a palavra `PROMOÇÃO`.

regex101.com

regex101.com

Padrão A: `kk*`

Padrão B: `URGENTE | PROMOCÃO`

O que cada símbolo faz:

- **^ (Circunflexo):** Representa o **início** da string ou da linha. Ele diz: "Comece a procurar exatamente aqui".
- **urgente** : É o texto literal que você quer casar.
- **\$ (Cifrão):** Representa o **fim** da string ou da linha. Ele diz: "A frase deve terminar imediatamente após a última letra 'e'".

- **JavaScript:** Perfeito para validações na web.

JavaScript



```
const regex = /^urgente$/  
console.log(regex.test("urgente")); // true
```

- **Python:** Usa o módulo `re`.

Python



```
import re
resultado = re.search(r'^urgente$', 'urgente')
```

Ciclo Completo

1. **A Receita (ER):** Escreva a Expressão Regular que define o padrão.
2. **O Rascunho (AFN):** Desenhe o Autômato Não-Determinístico (mais fácil de criar direto da ER).
3. **A Máquina Real (AFD):** Converta o AFN para um Autômato Determinístico (garantindo que cada estado tenha exatamente uma saída para cada símbolo).

Exercício 1: O Verificador de "Bits de Início"

Enunciado: Um sistema de rede recebe pacotes binários ($\Sigma = \{0, 1\}$). Ele só deve processar pacotes que **comecem obrigatoriamente com "01"**. Depois disso, qualquer sequência de 0s e 1s é permitida.

- **Exemplo Aceito:** 01, 01000, 01111.
- **Exemplo Rejeitado:** 10, 001, 1101.

Exercício 2: O Filtro de Comentários (Palavra "OI")

Enunciado: Uma rede social quer detectar mensagens que contenham a palavra "OI" em qualquer lugar do texto. Para simplificar, considere o alfabeto $\Sigma = \{O, I\}$. A mensagem pode ter outros caracteres antes ou depois, desde que o "OI" apareça pelo menos uma vez.

- **Exemplo Aceito:** OI, OOOI, OII, IOI.
- **Exemplo Rejeitado:** I, O, IIII, OOOO.